Московский государственный университет
имени М. В. Ломоносова

Факультет вычислительной математики и кибернетики
Кафедра исследования операций



Практикум на ЭВМ

Метод линеаризации

группа 411

Леонтьев Владимир Александрович

Москва, 2026

Содержание

1 Постановка задачи **3**

2 Теоретическое обоснование метода **4**

2.1 Необходимые условия оптимальности 4

2.2 Функция точного штрафа 4

2.3 Вспомогательная задача 5

2.4 Направление убывания функции штрафа 7

2.5 Алгоритм метода линеаризации 9

2.6 Сходимость метода 9

2.7 Иллюстративный пример 11

3 Практические аспекты **12**

3.1 Решение вспомогательной задачи 12

3.2 Замечания к алгоритму 13

3.3 Программная реализация 14

4 Эксперименты **15**

4.1 Сравнение с библиотечной функцией 15

4.2 Поведение метода 15

Заключение **18**

A Приложение **20**

A.1 Модуль `linearization.py` 20

A.2 Модуль `problems.py` 22

A.3 Модуль `experiments.py` 23

Список литературы **26**

1 Постановка задачи

Рассматривается задача нелинейного программирования

$$f(x) \to \min, \quad x \in X, \tag{1}$$

$$X = \big\{x \in \mathbb{R}^n : \ g_i(x) \le 0, \ i \in I; \quad g_i(x) = 0, \ i \in E\big\}, \tag{2}$$

где $I = \{1, \dots, m\}$ — множество номеров ограничений-неравенств, $E = \{m+1, \dots, s\}$ — множество номеров ограничений-равенств (любое из этих множеств может быть пустым), функции $f, g_1, \dots, g_s \colon \mathbb{R}^n \to \mathbb{R}$ непрерывно дифференцируемы. Через $f'(x)$, $g_i'(x)$ обозначаются градиенты функций, через $\langle \cdot, \cdot \rangle$ и $\|\cdot\|$ — стандартные скалярное произведение и евклидова норма в $\mathbb{R}^n$.

Никаких предположений о выпуклости функций f, g_i не делается, поэтому в общем случае речь может идти лишь о поиске точек, удовлетворяющих необходимым условиям локального минимума (стационарных точек задачи). Кроме того, не предполагается, что известна какая-либо допустимая точка: начальное приближение $x_0 \in \mathbb{R}^n$ произвольно.

Требуется изучить и программно реализовать *метод линеаризации* Б.Н. Пшеничного [3] для решения задачи (1)–(2):

- описать метод и привести теоретическое обоснование его сходимости;
- реализовать метод на ЭВМ;
- провести вычислительные эксперименты на тестовых задачах и проанализировать результаты.

Идея метода состоит в следующем. В текущей точке x_k целевая функция и ограничения заменяются своими линейными частями (отсюда название метода), к линеаризованной целевой функции добавляется квадратичный регуляризующий член, и решается получившаяся вспомогательная задача квадратичного программирования. Её решение задаёт направление движения, а длина шага выбирается из условия убывания специальной *функции точного штрафа*, которая одновременно учитывает значение целевой функции и степень нарушения ограничений.

2 Теоретическое обоснование метода

2.1 Необходимые условия оптимальности

Напомним необходимые условия экстремума, которые служат критерием «правильного» результата работы метода.

Определение 1. Функцией Лагранжа задачи (1)–(2) называется функция

$$L(x, \lambda, \mu) = f(x) + \sum_{i \in I} \lambda_i g_i(x) + \sum_{i \in E} \mu_i g_i(x), \qquad \lambda \in \mathbb{R}^m,\ \mu \in \mathbb{R}^{s-m}.$$

Определение 2. Точка $x^* \in \mathbb{R}^n$ называется *стационарной точкой* (точкой Каруша — Куна — Таккера) задачи (1)–(2), если найдутся множители $\lambda^* \in \mathbb{R}^m$, $\mu^* \in \mathbb{R}^{s-m}$ такие, что

$$f'(x^*) + \sum_{i \in I} \lambda_i^* g_i'(x^*) + \sum_{i \in E} \mu_i^* g_i'(x^*) = 0, \tag{3}$$

$$g_i(x^*) \le 0, \quad \lambda_i^* \ge 0, \quad \lambda_i^* g_i(x^*) = 0, \quad i \in I; \qquad g_i(x^*) = 0, \quad i \in E. \tag{4}$$

Теорема 1 ([2; 5])**.** Пусть x^* — точка локального минимума задачи (1)–(2), функции f, g_i непрерывно дифференцируемы в окрестности x^*, и градиенты активных в точке x^* ограничений $\{g_i'(x^*) : i \in E \text{ или } i \in I,\ g_i(x^*) = 0\}$ линейно независимы. Тогда x^* — стационарная точка задачи, т. е. выполнены условия (3)–(4).

Без условия регулярности теорема 1 верна в форме принципа Лагранжа с множителем $\lambda_0 \ge 0$ при $f'(x^*)$ [5]; в дальнейшем нас будет интересовать именно случай $\lambda_0 = 1$, т. е. условия (3)–(4). Для выпуклой задачи (функции f, g_i, $i \in I$ выпуклы, g_i, $i \in E$ аффинны) эти условия являются и достаточными для глобального минимума [2; 6].

2.2 Функция точного штрафа

Поскольку начальная точка может быть недопустимой, метод должен одновременно уменьшать значение целевой функции и невязку ограничений. Меру нарушения ограничений в точке x зададим функцией

$$F(x) = \max\bigl\{0;\ g_1(x), \ldots, g_m(x);\ |g_{m+1}(x)|, \ldots, |g_s(x)|\bigr\}. \tag{5}$$

Очевидно, $F(x) \ge 0$ для всех x, и $F(x) = 0$ тогда и только тогда, когда $x \in X$. При фиксированном $N > 0$ введём *функцию точного штрафа*

$$\Phi_N(x) = f(x) + NF(x).$$

Функция Φ_N непрерывна, но, вообще говоря, не дифференцируема (в тех точках, где максимум в (5) достигается на нескольких функциях). Термин «точный» объясняется тем, что при достаточно большом, но *конечном* N минимум Φ_N на всём пространстве совпадает с решением исходной задачи [2; 3]. Для выпуклого случая это легко показать.

Утверждение 1. Пусть функции f, g_i, $i \in I$ выпуклы и дифференцируемы, g_i, $i \in E$ аффинны, x^* — стационарная точка задачи с множителями λ^*, μ^* и

$$N \geq \sum_{i \in I} \lambda_i^* + \sum_{i \in E} |\mu_i^*|.$$

Тогда x^* — точка глобального минимума функции Φ_N на $\mathbb{R}^n$.

Доказательство. Возьмём произвольное $x \in \mathbb{R}^n$. По выпуклости f и условию (3)

$$f(x) \geq f(x^*) + \langle f'(x^*),\, x - x^* \rangle = f(x^*) - \sum_{i \in I} \lambda_i^* \langle g_i'(x^*),\, x - x^* \rangle - \sum_{i \in E} \mu_i^* \langle g_i'(x^*),\, x - x^* \rangle.$$

Для $i \in I$ из выпуклости g_i, неотрицательности λ_i^* и условия дополняющей нежёсткости $\lambda_i^* g_i(x^*) = 0$ получаем $-\lambda_i^* \langle g_i'(x^*),\, x - x^* \rangle \geq -\lambda_i^* \big(g_i(x) - g_i(x^*)\big) = -\lambda_i^* g_i(x)$. Для $i \in E$ в силу аффинности и $g_i(x^*) = 0$ имеем $\langle g_i'(x^*),\, x - x^* \rangle = g_i(x)$. Следовательно,

$$\begin{aligned} f(x) &\geq f(x^*) - \sum_{i \in I} \lambda_i^* g_i(x) - \sum_{i \in E} \mu_i^* g_i(x) \\ &\geq f(x^*) - \Big(\sum_{i \in I} \lambda_i^* + \sum_{i \in E} |\mu_i^*| \Big) F(x) \\ &\geq f(x^*) - N F(x), \end{aligned}$$

так как $g_i(x) \leq F(x)$ при $i \in I$ и $|g_i(x)| \leq F(x)$ при $i \in E$. Итак, $\Phi_N(x) \geq f(x^*) = \Phi_N(x^*)$, поскольку $F(x^*) = 0$. □

В невыпуклом случае аналогичное утверждение верно локально: если в точке x^* выполнены достаточные условия локального минимума второго порядка и N строго больше суммы модулей множителей Лагранжа, то x^* — точка локального минимума Φ_N [2; 4]. Таким образом, задача условной минимизации сводится к безусловной минимизации негладкой функции Φ_N, и метод линеаризации можно рассматривать как специальный метод спуска для Φ_N.

2.3 Вспомогательная задача

Зафиксируем параметр $\delta > 0$ и для точки $x \in \mathbb{R}^n$ определим множество δ-*активных* ограничений-неравенств

$$I_\delta(x) = \{ i \in I :\ g_i(x) \geq F(x) - \delta \}. \tag{6}$$

Это те неравенства, которые в точке x нарушены «почти максимально» или близки к активным. Ограничения-равенства учитываются всегда.

Пусть x_k — текущее приближение. Направление p_k определим как решение вспомогательной задачи

$$\begin{gathered}\langle f'(x_k),\, p\rangle + \tfrac{1}{2}\|p\|^2 \to \min_{p\in\mathbb{R}^n},\\ g_i(x_k) + \langle g_i'(x_k),\, p\rangle \le 0, \quad i \in I_\delta(x_k); \qquad g_i(x_k) + \langle g_i'(x_k),\, p\rangle = 0, \quad i \in E.\end{gathered} \tag{7}$$

Ограничения задачи (7) — это линеаризации ограничений исходной задачи в точке x_k, а целевая функция — линейная часть приращения f, «регуляризованная» квадратичным членом. Без этого члена линеаризованная задача могла бы не иметь решения (линейная функция на неограниченном многограннике), а найденная точка могла бы оказаться слишком далеко от x_k, где линейные приближения уже неточны.

Утверждение 2. Если допустимое множество задачи (7) непусто, то она имеет единственное решение p_k, и найдутся множители $\lambda_i^k \ge 0$, $i \in I_\delta(x_k)$, и μ_i^k, $i \in E$, такие, что

$$f'(x_k) + p_k + \sum_{i\in I_\delta(x_k)} \lambda_i^k g_i'(x_k) + \sum_{i\in E} \mu_i^k g_i'(x_k) = 0, \tag{8}$$

$$\lambda_i^k \big(g_i(x_k) + \langle g_i'(x_k),\, p_k\rangle\big) = 0, \quad i \in I_\delta(x_k). \tag{9}$$

Обратно, если вектор p_k, допустимый в (7), удовлетворяет условиям (8)–(9) с некоторыми $\lambda^k \ge 0$, μ^k, то p_k — решение задачи (7).

Доказательство. Целевая функция задачи (7) сильно выпукла, допустимое множество — непустой замкнутый многогранник. Сильно выпуклая непрерывная функция достигает на непустом замкнутом выпуклом множестве минимума в единственной точке [3; 6]. Поскольку все ограничения задачи (7) линейны, условие регулярности выполнено автоматически, и условия Каруша — Куна — Таккера (8)–(9) необходимы; в силу выпуклости задачи они и достаточны [2; 6]. □

Для удобства положим $\lambda_i^k = 0$ при $i \in I \setminus I_\delta(x_k)$ и обозначим

$$\Lambda_k = \sum_{i\in I} \lambda_i^k + \sum_{i\in E} |\mu_i^k|. \tag{10}$$

Замечание 1. В двух частных случаях вспомогательная задача приводит к хорошо известным методам. Если ограничений нет, то $p_k = -f'(x_k)$, и метод линеаризации превращается в градиентный метод с дроблением шага. Если все ограничения линейны и $x_k \in X$, то точка $\bar{x}_k = x_k + p_k$ есть проекция точки $x_k - f'(x_k)$ на множество, задаваемое δ-активными ограничениями, т. е. получается вариант метода проекции градиента [3].

Следующая лемма показывает, что равенство $p_k = 0$ является критерием стационарности.

Лемма 1. Пусть задача (7) в точке x_k разрешима. Тогда $p_k = 0$ в том и только в том случае, когда x_k — стационарная точка задачи (1)–(2).

Доказательство. Пусть $p_k = 0$. Тогда из допустимости p_k в (7) имеем $g_i(x_k) \le 0$ при $i \in I_\delta(x_k)$ и $g_i(x_k) = 0$ при $i \in E$. Максимум в (5) достигается либо на нуле, либо на некотором $|g_i|$, $i \in E$ (тогда он равен нулю), либо на некотором g_i, $i \in I$; в последнем случае $g_i(x_k) = F(x_k) \ge F(x_k) - \delta$, т. е. $i \in I_\delta(x_k)$, и значит $F(x_k) = g_i(x_k) \le 0$. Во всех случаях $F(x_k) = 0$, т. е. $x_k \in X$. Условия (8)–(9) при $p_k = 0$ (с $\lambda_i^k = 0$ вне $I_\delta(x_k)$) в точности совпадают с (3)–(4).

Обратно, пусть x_k — стационарная точка с множителями λ^*, μ^*. Тогда $F(x_k) = 0$, и если $i \in I \setminus I_\delta(x_k)$, то $g_i(x_k) < -\delta < 0$, откуда $\lambda_i^* = 0$ по условию дополняющей нежёсткости. Значит, вектор $p = 0$ допустим в (7) и вместе с множителями λ_i^*, $i \in I_\delta(x_k)$, μ^* удовлетворяет (8)–(9). По утверждению 2 $p_k = 0$. □

2.4 Направление убывания функции штрафа

Основное свойство направления p_k составляет следующая оценка.

Лемма 2. Для решения p_k задачи (7) и величины Λ_k из (10) справедливо неравенство

$$\langle f'(x_k),\, p_k\rangle \le -\|p_k\|^2 + \Lambda_k F(x_k).$$

Доказательство. Умножим (8) скалярно на p_k:

$$\langle f'(x_k),\, p_k\rangle = -\|p_k\|^2 - \sum_{i\in I_\delta(x_k)} \lambda_i^k \langle g_i'(x_k),\, p_k\rangle - \sum_{i\in E} \mu_i^k \langle g_i'(x_k),\, p_k\rangle.$$

Из (9) следует $\lambda_i^k \langle g_i'(x_k),\, p_k\rangle = -\lambda_i^k g_i(x_k)$, а из ограничений-равенств задачи (7) — $\langle g_i'(x_k),\, p_k\rangle = -g_i(x_k)$, $i \in E$. Поэтому

$$\langle f'(x_k),\, p_k\rangle = -\|p_k\|^2 + \sum_{i\in I} \lambda_i^k g_i(x_k) + \sum_{i\in E} \mu_i^k g_i(x_k) \le -\|p_k\|^2 + \Lambda_k F(x_k),$$

поскольку $\lambda_i^k \ge 0$, $g_i(x_k) \le F(x_k)$ и $\mu_i^k g_i(x_k) \le |\mu_i^k|\, F(x_k)$. □

Далее будем предполагать, что градиенты функций f, g_i удовлетворяют условию Липшица с общей константой L:

$$\|f'(x) - f'(y)\| \le L\|x - y\|, \quad \|g_i'(x) - g_i'(y)\| \le L\|x - y\|, \quad x, y \in \mathbb{R}^n,\ i = 1, \ldots, s. \qquad (11)$$

Хорошо известно [3], что тогда для любой из этих функций φ

$$\bigl|\varphi(x+h)-\varphi(x)-\langle\varphi'(x),\,h\rangle\bigr|\le\tfrac{L}{2}\|h\|^2,\qquad x,h\in\mathbb{R}^n. \tag{12}$$

Лемма 3. Пусть выполнено (11), задача (7) разрешима, $\Lambda_k\le N$, $\varepsilon\in(0,1)$, и пусть числа P, G, Φ таковы, что $\|p_k\|\le P$, $\|g_i'(x_k)\|\le G$ при всех $i\in I$, $F(x_k)\le\Phi$. Тогда для всех $\alpha\in(0,\bar\alpha]$, где

$$\bar\alpha=\min\Bigl\{1;\ \frac{2(1-\varepsilon)}{(1+N)L};\ \frac{\delta}{GP+\Phi}\Bigr\}, \tag{13}$$

выполняется неравенство

$$\Phi_N(x_k+\alpha p_k)\le\Phi_N(x_k)-\varepsilon\alpha\|p_k\|^2. \tag{14}$$

Доказательство. Обозначим $x=x_k$, $p=p_k$, $r=\frac{L}{2}\alpha^2\|p\|^2$ и оценим $F(x+\alpha p)$, рассмотрев все функции под знаком максимума в (5).

1) Пусть $i\in I_\delta(x)$. По (12) и ограничению задачи (7)

$$\begin{aligned} g_i(x+\alpha p)&\le g_i(x)+\alpha\langle g_i'(x),\,p\rangle+r\\ &=(1-\alpha)g_i(x)+\alpha\bigl(g_i(x)+\langle g_i'(x),\,p\rangle\bigr)+r\le(1-\alpha)F(x)+r. \end{aligned}$$

2) Пусть $i\in I\setminus I_\delta(x)$, т. е. $g_i(x)<F(x)-\delta$. Тогда

$$g_i(x+\alpha p)\le g_i(x)+\alpha GP+r<F(x)-\delta+\alpha GP+r\le(1-\alpha)F(x)+r,$$

так как $\alpha(GP+F(x))\le\alpha(GP+\Phi)\le\delta$ в силу выбора $\bar\alpha$.

3) Пусть $i\in E$. Так как $\langle g_i'(x),\,p\rangle=-g_i(x)$, из (12) получаем $g_i(x+\alpha p)=(1-\alpha)g_i(x)+\theta$ с $|\theta|\le r$, откуда $|g_i(x+\alpha p)|\le(1-\alpha)F(x)+r$.

4) Наконец, $0\le(1-\alpha)F(x)+r$ при $\alpha\le1$.

Таким образом, $F(x+\alpha p)\le(1-\alpha)F(x)+r$. Далее, по (12) и лемме 2

$$f(x+\alpha p)\le f(x)+\alpha\langle f'(x),\,p\rangle+r\le f(x)-\alpha\|p\|^2+\alpha\Lambda_kF(x)+r.$$

Складывая, получаем

$$\begin{aligned} \Phi_N(x+\alpha p)&\le\Phi_N(x)-\alpha\|p\|^2-\alpha(N-\Lambda_k)F(x)+(1+N)r\\ &\le\Phi_N(x)-\alpha\|p\|^2\Bigl(1-\frac{(1+N)L\alpha}{2}\Bigr). \end{aligned}$$

При $\alpha\le2(1-\varepsilon)/((1+N)L)$ скобка не меньше ε, что и даёт (14). □

Из леммы 3, в частности, следует, что при $p_k\ne0$ направление p_k является направлением строгого убывания функции Φ_N в точке x_k, причём неравенство (14) выполняется для всех достаточно малых шагов. Это и делает возможным выбор шага дроблением.

2.5 Алгоритм метода линеаризации

Алгоритм (метод линеаризации)**.**

1. *Инициализация.* Задать начальную точку $x_0 \in \mathbb{R}^n$ (не обязательно допустимую), параметры $N > 0$, $\delta > 0$, $\varepsilon \in (0,1)$ и точность $\tau > 0$. Положить $k := 0$.

2. *Активные ограничения.* Вычислить $F(x_k)$ по формуле (5) и множество $I_\delta(x_k)$ по формуле (6).

3. *Направление.* Решить задачу квадратичного программирования (7); найти её решение p_k и множители λ^k, μ^k.

4. *Критерий остановки.* Если $\|p_k\| \le \tau$ и $F(x_k) \le \tau$, то завершить работу, приняв x_k за приближённую стационарную точку, а λ^k, μ^k — за приближённые множители Лагранжа.

5. *Шаг.* Положить $\alpha := 1$. Пока $\Phi_N(x_k + \alpha p_k) > \Phi_N(x_k) - \varepsilon\alpha\|p_k\|^2$, полагать $\alpha := \alpha/2$. Полученное значение обозначить α_k.

6. *Переход.* Положить $x_{k+1} := x_k + \alpha_k p_k$, $k := k+1$ и перейти к шагу 2.

По лемме 3 цикл на шаге 5 завершается за конечное число дроблений, и найденный шаг удовлетворяет оценке $\alpha_k \ge \min\{1, \bar\alpha/2\}$: действительно, если шаг α был отвергнут, то $\alpha > \bar\alpha$, а следующий пробный шаг равен $\alpha/2 > \bar\alpha/2$.

2.6 Сходимость метода

Теорема 2. Пусть функции f, $g_1, \dots, g_s$ непрерывно дифференцируемы и их градиенты удовлетворяют условию Липшица (11); множество

$$\Omega = \{x \in \mathbb{R}^n : \ \Phi_N(x) \le \Phi_N(x_0)\}$$

ограничено; для каждой точки $x \in \Omega$ вспомогательная задача (7), построенная в точке x, разрешима, и сумма модулей её множителей Лагранжа не превосходит N. Тогда при $\tau = 0$ алгоритм из п. 2.5 либо останавливается в стационарной точке задачи (1)–(2), либо строит бесконечную последовательность $\{x_k\}$, для которой

a) $\Phi_N(x_{k+1}) < \Phi_N(x_k)$ при всех k;

b) $\|p_k\| \to 0$ при $k \to \infty$;

с) любая предельная точка x^* последовательности $\{x_k\}$ (а они существуют) допустима и является стационарной точкой задачи (1)–(2).

Доказательство. Если на некотором шаге $p_k = 0$, то по лемме 1 точка x_k стационарна. Пусть $p_k \neq 0$ при всех k.

Равномерные оценки. Если $x_k \in \Omega$, то по лемме 3 (которую мы применим с константами, указанными ниже) $\Phi_N(x_{k+1}) \leq \Phi_N(x_k) - \varepsilon\alpha_k\|p_k\|^2 < \Phi_N(x_k)$, так что $x_{k+1} \in \Omega$; по индукции вся последовательность лежит в Ω и выполнено утверждение а). Множество Ω замкнуто (как множество уровня непрерывной функции) и ограничено, т. е. компактно. На нём непрерывные функции $\|f'(x)\|$, $\|g_i'(x)\|$ и $F(x)$ ограничены некоторыми константами G_0, G, Φ. Из (8) и условия $\Lambda_k \leq N$ получаем $\|p_k\| \leq \|f'(x_k)\| + \Lambda_k \max_i\|g_i'(x_k)\| \leq G_0 + NG =: P$. Следовательно, величина $\bar\alpha$ из (13) не зависит от k, и $\alpha_k \geq \underline{\alpha} := \min\{1, \bar\alpha/2\} > 0$.

Утверждение б). Функция Φ_N ограничена снизу на компакте Ω некоторой константой Φ_*. Суммируя неравенства (14), получаем

$$\varepsilon\underline{\alpha}\sum_{k=0}^{K}\|p_k\|^2 \leq \sum_{k=0}^{K}\big(\Phi_N(x_k) - \Phi_N(x_{k+1})\big) \leq \Phi_N(x_0) - \Phi_*$$

при любом K. Значит, ряд $\sum\|p_k\|^2$ сходится и $\|p_k\| \to 0$.

Утверждение в). Существование предельных точек следует из компактности Ω. Пусть $x_{k_j} \to x^*$. Поскольку подмножеств множества I конечное число, а множители ограничены ($\Lambda_k \leq N$), можно, переходя к подпоследовательности, считать, что $I_\delta(x_{k_j}) = J$ для всех j и $\lambda^{k_j} \to \lambda^*$, $\mu^{k_j} \to \mu^*$. Перейдём к пределу при $j \to \infty$ в (8), в (9) и в ограничениях задачи (7), учитывая непрерывность градиентов и $p_{k_j} \to 0$:

$$f'(x^*) + \sum_{i\in I}\lambda_i^* g_i'(x^*) + \sum_{i\in E}\mu_i^* g_i'(x^*) = 0,$$

$$\begin{aligned} &g_i(x^*) \leq 0,\ \lambda_i^* \geq 0,\ \lambda_i^* g_i(x^*) = 0, && i \in J,\\ &\lambda_i^* = 0, && i \in I \setminus J,\\ &g_i(x^*) = 0, && i \in E. \end{aligned}$$

Остаётся показать, что $g_i(x^*) \leq 0$ и при $i \in I \setminus J$. Значения $g_i(x)$ при $i \in I \setminus I_\delta(x)$ меньше $F(x) - \delta$ и потому не влияют на максимум в (5), так что $F(x_{k_j}) = \max\{0;\ g_i(x_{k_j}),\ i \in J;\ |g_i(x_{k_j})|,\ i \in E\}$. Переходя к пределу, получаем $F(x^*) = \max\{0;\ g_i(x^*),\ i \in J;\ |g_i(x^*)|,\ i \in E\} = 0$, т. е. $x^* \in X$ и, в частности, $g_i(x^*) \leq 0$ при всех $i \in I$. Таким образом, x^* удовлетворяет условиям (3)–(4). □

Замечание 2. Условие разрешимости вспомогательной задачи в точках Ω существенно: если линеаризованные ограничения несовместны, направление p_k не определено. Для допустимых точек и точек, близких к допустимому множеству, совместность гарантируется условиями регулярности ограничений (например, линейной независимостью градиентов активных ограничений, как в теореме 1, или условием Мангасарьяна — Фромовица) [4].

Замечание 3 (о выборе параметра N)**.** Значение N, при котором $\Lambda_k \le N$, заранее, как правило, неизвестно. На практике используют адаптивное правило [3; 4]: после решения задачи (7) проверяют условие $\Lambda_k \le N_k$ и при его нарушении увеличивают параметр, например $N_{k+1} := \max\{2N_k,\ \Lambda_k\}$. Если множители вспомогательных задач ограничены, то параметр увеличивается лишь конечное число раз, после чего работает теорема 2.

Замечание 4 (о выборе параметра δ)**.** При $\delta = +\infty$ во вспомогательную задачу включаются все ограничения; малое δ уменьшает размерность задачи (7), но, как видно из (13), может приводить к более мелким шагам.

Замечание 5 (о скорости сходимости)**.** Метод линеаризации в описанной форме является методом первого порядка и в общем случае сходится не быстрее градиентных методов. Если заменить квадратичный член $\frac{1}{2}\|p\|^2$ в (7) на $\frac{1}{2}\langle H_k p,\, p\rangle$ с положительно определённой матрицей H_k, аппроксимирующей гессиан функции Лагранжа по x, то получится метод последовательного квадратичного программирования (SQP), обладающий при естественных предположениях локальной сверхлинейной сходимостью [2; 4]. Обоснование глобальной сходимости при этом проводится аналогично, с помощью той же функции штрафа Φ_N.

2.7 Иллюстративный пример

Пример 1. Рассмотрим задачу

$$f(x) = x_1^2 + x_2^2 \to \min, \qquad g_1(x) = 1 - x_1 - x_2 \le 0.$$

Её решение $x^* = (1/2, 1/2)$, множитель Лагранжа $\lambda^* = 1$. Возьмём недопустимую начальную точку $x_0 = (0, 0)$ и параметры $N = 2$, $\varepsilon = 1/2$, $\delta = 1$.

Итерация 0. $F(x_0) = 1$, $I_\delta(x_0) = \{1\}$, $f'(x_0) = (0, 0)$, $g_1'(x_0) = (-1, -1)$. Вспомогательная задача: $\frac{1}{2}\|p\|^2 \to \min$ при $1 - p_1 - p_2 \le 0$. Её решение — проекция нуля на полуплоскость: $p_0 = (1/2, 1/2)$, из (8) $\lambda^0 = 1/2 \le N$. Проверим шаг $\alpha = 1$: $\Phi_N(x_0) = 0 + 2 \cdot 1 = 2$, $\Phi_N(x_0 + p_0) = 1/2 + 0 = 1/2 \le 2 - \frac{1}{2} \cdot 1 \cdot \frac{1}{2}$. Шаг принят: $x_1 = (1/2, 1/2)$.

Итерация 1. $F(x_1) = 0$, $I_\delta(x_1) = \{1\}$, $f'(x_1) = (1, 1)$. Вспомогательная задача: $p_1 + p_2 + \frac{1}{2}\|p\|^2 \to \min$ при $-p_1 - p_2 \le 0$. На допустимом множестве $p_1 + p_2 \ge 0$, поэтому целевая функция не меньше нуля и равна нулю лишь при $p = 0$. Итак, решение вспомогательной задачи — нулевой вектор, множитель $\lambda^1 = 1$, и по лемме 1 точка $x_1 = x^*$ стационарна; метод останавливается.

Пример показывает характерную особенность метода: первый шаг из недопустимой точки направлен в первую очередь на устранение невязки ограничений, а множители вспомогательной задачи в пределе совпадают с множителями Лагранжа исходной задачи.

3 Практические аспекты

3.1 Решение вспомогательной задачи

Основная вычислительная работа на каждой итерации — решение задачи квадратичного программирования (7). Поскольку матрица квадратичной части единичная, задача допускает простое сведение к задаче о наименьшем расстоянии. Обозначим $c = f'(x_k)$ и сделаем замену $z = p + c$. Тогда $\langle c,\, p\rangle + \frac{1}{2}\|p\|^2 = \frac{1}{2}\|z\|^2 - \frac{1}{2}\|c\|^2$, т. е. решение p_k задачи (7) есть $p_k = z_k - c$, где z_k — проекция нуля на многогранник линеаризованных ограничений:

$$\tfrac{1}{2}\|z\|^2 \to \min, \qquad Gz \ge h. \tag{15}$$

Здесь каждому ограничению-неравенству $i \in I_\delta(x_k)$ отвечает строка $-g_i'(x_k)$ матрицы G и компонента $g_i(x_k) - \langle g_i'(x_k),\, c\rangle$ вектора h, а каждое ограничение-равенство $i \in E$ записывается в виде двух неравенств со строками $\mp g_i'(x_k)$ и правыми частями $\pm\big(g_i(x_k) - \langle g_i'(x_k),\, c\rangle\big)$. Обозначим через q число строк матрицы G.

Задача (15) (в англоязычной литературе — LDP, least distance programming) сводится к задаче неотрицательных наименьших квадратов (NNLS), для которой имеется конечный алгоритм Лоусона — Хенсона [7].

Утверждение 3. Пусть $E = \begin{pmatrix} G^\top \\ h^\top \end{pmatrix} \in \mathbb{R}^{(n+1)\times q}$, $e = (0, \ldots, 0, 1)^\top \in \mathbb{R}^{n+1}$, u — решение задачи $\|Eu - e\|^2 \to \min$, $u \ge 0$, и $r = Eu - e$. Тогда

1) если $r = 0$, то система $Gz \ge h$ несовместна;

2) если $r \ne 0$, то $r_{n+1} < 0$, вектор $z = G^\top w$, где $w = u/(-r_{n+1}) \ge 0$, является решением задачи (15), а w — соответствующим вектором множителей Лагранжа.

Доказательство. Условия оптимальности задачи NNLS имеют вид $E^\top r \geq 0$, $\langle u,\, E^\top r\rangle = 0$ [7].

1) Если $r = 0$, то $G^\top u = 0$ и $\langle h,\, u\rangle = 1$. Если бы нашлось z с $Gz \geq h$, то, так как $u \geq 0$, было бы $0 = \langle G^\top u,\, z\rangle = \langle u,\, Gz\rangle \geq \langle u,\, h\rangle = 1$ — противоречие.

2) Из условия дополняющей нежёсткости $\|r\|^2 = \langle r,\, Eu\rangle - \langle r,\, e\rangle = -r_{n+1}$, поэтому $r_{n+1} < 0$. Первые n компонент вектора r равны $G^\top u = -r_{n+1} z$, так что $E^\top r = G(-r_{n+1} z) + h\, r_{n+1} = -r_{n+1}(Gz - h) \geq 0$, т. е. z допустим. Далее $w \geq 0$, $z = G^\top w$ и $\langle w,\, Gz - h\rangle = \langle u,\, E^\top r\rangle / r_{n+1}^2 = 0$. Это условия Каруша — Куна — Таккера для выпуклой задачи (15) с линейными ограничениями, которые достаточны для оптимальности. □

Сравнивая условие $z_k = G^\top w$ с условием (8), записанным в виде $f'(x_k) + p_k = -\sum \lambda_i^k g_i'(x_k) - \sum \mu_i^k g_i'(x_k)$, получаем множители вспомогательной задачи: λ_i^k совпадает с компонентой w, отвечающей неравенству i, а $\mu_i^k = w_i^+ - w_i^-$, где $w_i^\pm$ — компоненты w, отвечающие паре неравенств, которыми заменено равенство i. Случай $r = 0$ означает, что линеаризованные ограничения несовместны и предположения теоремы 2 нарушены; программа в этом случае сообщает об ошибке.

3.2 Замечания к алгоритму

1. *Погрешность вычислений.* В машинной арифметике равенство $p_k = 0$ и условие $F(x_k) = 0$ не достигаются точно, поэтому используется критерий остановки с параметром $\tau > 0$ (шаг 4 алгоритма). Кроме того, невязка задачи NNLS сравнивается с нулём с допуском 10^{-12}.

2. *Ограничение на дробление шага.* Если предположения теоремы 2 нарушены (например, N слишком мало), цикл дробления шага может не завершиться за разумное число шагов. Поэтому дробление прекращается, как только α становится меньше $\alpha_{\min} = 10^{-12}$, а общее число итераций ограничено величиной $k_{\max}$. Невыполнение критерия остановки за $k_{\max}$ итераций сигнализирует о неудачном выборе параметров.

3. *Выбор параметра N.* Реализовано адаптивное правило из замечания 3: после решения вспомогательной задачи вычисляется Λ_k и при $\Lambda_k > N$ полагается $N := \max\{2N, \Lambda_k\}$. Пока N меняется, монотонность функции Φ_N вдоль траектории не гарантируется, но после последнего увеличения параметра работает теорема 2.

4. *Ограничения-равенства.* Замена равенства парой неравенств делает задачу (15) вырожденной (пара строк $\pm g_i'$ линейно зависима), и вектор w определяется неоднозначно. Однако z_k единственно, а разность $w_i^+ - w_i^-$ определена однозначно при линейной независимости градиентов ограничений, вошедших во вспомогательную задачу.

5. *Трудоёмкость.* На каждой итерации вычисляются градиенты f и всех g_i (одна матрица Якоби), решается одна задача NNLS размера $(n{+}1){\times}q$, $q \le m{+}2(s{-}m)$, и при дроблении шага вычисляются значения f и g в пробных точках.

3.3 Программная реализация

Метод реализован на языке Python 3 с использованием библиотек NumPy и SciPy. Исходный код проекта, включая результаты экспериментов, доступен в GitHub-репозитории[1]. Программа состоит из трёх модулей:

- `linearization.py` — реализация метода: вычисление функций F и Φ_N, решение задачи (15) через NNLS (функция `scipy.optimize.nnls`, реализующая алгоритм Лоусона — Хенсона), построение направления p_k и множителей λ^k, μ^k, основной цикл алгоритма из п. 2.5 (листинг в приложении А.1);

- `problems.py` — тестовые задачи; задача задаётся функциями f, f', вектор-функцией $g = (g_1, \ldots, g_s)$, её матрицей Якоби и числом неравенств m (приложение А.2);

- `experiments.py` — проведение экспериментов, проверка ответа библиотечной функцией и построение графиков (приложение А.3).

Работоспособность программы проверяется сравнением с библиотечной функцией `scipy.optimize.minimize` (метод SLSQP — последовательное квадратичное программирование). Каждая тестовая задача решается обоими способами из одной и той же начальной точки, после чего сравниваются найденные точки и значения целевой функции; ответ считается подтверждённым, если метод линеаризации выполнил критерий остановки и $\|x_K - x_{\text{SLSQP}}\| < 10^{-5}$.

Функция `linearization` принимает задачу, начальную точку x_0 и параметры N, δ, ε, τ, $k_{\max}$, $\alpha_{\min}$ и возвращает найденную точку, множители Лагранжа, итоговое значение N, число итераций, признак выполнения критерия остановки и протокол итераций (для каждой итерации — x_k, $f(x_k)$, $F(x_k)$, $\|p_k\|$, $|I_\delta(x_k)|$, Λ_k, N, α_k).

[1] `https://github.com/LeonteVVA/linearization-method` (дата обращения: 16.10.2026)

4 Эксперименты

Во всех экспериментах, если не оговорено иное, использовались параметры $\varepsilon = 0{,}5$, $\delta = 1$, $\tau = 10^{-6}$, $k_{\max} = 10^4$ и адаптивный выбор N. Рассматривались следующие задачи.

Пример 1 (выпуклая задача из п. 2.7)**.** $f(x) = x_1^2 + x_2^2 \to \min$, $1 - x_1 - x_2 \le 0$; $x_0 = (0, 0)$, $N_0 = 2$. Решение $x^* = (0{,}5;\ 0{,}5)$, $\lambda^* = 1$.

Пример 2 (неравенство и равенство)**.** $f(x) = (x_1 - 2)^2 + (x_2 - 1)^2 \to \min$, $\frac{1}{4}x_1^2 + x_2^2 - 1 \le 0$, $x_1 - 2x_2 + 1 = 0$; $x_0 = (-2, 2)$, $N_0 = 1$. Решение $x^* = \left(\frac{\sqrt{7}-1}{2},\ \frac{\sqrt{7}+1}{4}\right) \approx (0{,}822876;\ 0{,}911438)$, $f(x^*) \approx 1{,}393465$.

Пример 3 (невыпуклое допустимое множество)**.** $f(x) = x_1 + x_2 \to \min$, $x_1^2 + x_2^2 - 2 = 0$; $x_0 = (2;\ 0{,}5)$, $N_0 = 1$. Решение $x^* = (-1, -1)$, $\mu^* = 1/2$.

Пример 4 (овражная функция)**.** $f(x) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2 \to \min$, $x_1^2 + x_2^2 - 1 \le 0$; $x_0 = (-1{,}5;\ 1{,}5)$, $N_0 = 1$. Безусловный минимум функции Розенброка $(1, 1)$ недопустим, решение лежит на границе круга.

Пример 5 (задача 71 из сборника [1])**.** $f(x) = x_1 x_4 (x_1 + x_2 + x_3) + x_3 \to \min$ при ограничениях $25 - x_1 x_2 x_3 x_4 \le 0$, $1 \le x_j \le 5$, $j = 1, \ldots, 4$, $x_1^2 + x_2^2 + x_3^2 + x_4^2 = 40$ ($n = 4$, $m = 9$, $s - m = 1$); $x_0 = (1, 5, 5, 1)$, $N_0 = 1$. Известное решение $x^* \approx (1;\ 4{,}7430;\ 3{,}8211;\ 1{,}3794)$, $f(x^*) \approx 17{,}014017$ [1].

4.1 Сравнение с библиотечной функцией

В таблице 1 сравниваются результаты метода линеаризации (x_K — последняя точка) и функции `scipy.optimize.minimize` с методом SLSQP (x_{SLSQP}). Во всех примерах критерий остановки выполнен, найденные точки и значения целевой функции совпадают с библиотечным решением (и с известными решениями) с точностью, согласованной с $\tau = 10^{-6}$, т. е. программа работает правильно.

4.2 Поведение метода

Найденные множители Лагранжа: в примере 2 $\lambda_1 \approx 1{,}8466$, $\mu_2 \approx 1{,}5945$; в примере 3 $\mu_1 = 0{,}5$; в примере 4 $\lambda_1 \approx 0{,}1215$; в примере 5 множители при ограничениях $25 - x_1 x_2 x_3 x_4 \le 0$ и $1 - x_1 \le 0$ равны 0,5523 и 1,0879, множитель при равенстве — 0,1615, остальные равны

Таблица 1: Сравнение метода линеаризации с `scipy.optimize.minimize` (SLSQP)

Пример	Итераций	$f(x_K)$	$f(x_{\text{SLSQP}})$	$\|x_K - x_{\text{SLSQP}}\|$	$F(x_K)$
1	1	0,500000	0,500000	$4 \cdot 10^{-16}$	$6 \cdot 10^{-17}$
2	6	1,393465	1,393465	$2 \cdot 10^{-11}$	$2 \cdot 10^{-11}$
3	7	−2,000000	−2,000000	$3 \cdot 10^{-7}$	$8 \cdot 10^{-7}$
4	2659	0,045675	0,045675	$2 \cdot 10^{-9}$	0
5	19	17,014017	17,014017	$7 \cdot 10^{-7}$	$2 \cdot 10^{-11}$

нулю. Во всех примерах итоговое значение параметра штрафа ($N_K = 2$; 3,5; 1; 4; 2,88 соответственно) не меньше Λ_K, что согласуется с утверждением 1.

Траектории метода для двумерных задач показаны на рис. 1 (закрашено множество, задаваемое неравенствами, линией показано множество, задаваемое равенством). Видно, что из недопустимой начальной точки метод за несколько шагов выходит на допустимое множество, после чего движется вдоль его границы. В примере 4 после выхода на границу круга метод медленно движется по дну «оврага» функции Розенброка, совершая мелкие зигзагообразные шаги.

На рис. 2 показано изменение $\|p_k\|$ и $F(x_k)$ по итерациям (по оси абсцисс — линейная шкала до $k = 10$ и логарифмическая далее; нулевые значения F условно изображены на уровне 10^{-16}). В примерах 2 и 3 в конце процесса принимается единичный шаг и сходимость сверхлинейная. Это объясняется спецификой задач: в примере 2 решение — точка пересечения двух активных ограничений, и метод совпадает с методом Ньютона для системы $g_1(x) = 0$, $g_2(x) = 0$; в примере 3 гессиан функции Лагранжа в решении равен $2\mu^* I = I$, т.е. совпадает с матрицей квадратичного члена $\frac{1}{2}\|p\|^2$ задачи (7), и метод линеаризации работает как метод SQP. В примере 5 невязка $F(x_k)$ также убывает быстро, а величина $\|p_k\|$ — линейно. Для примера 4, где решение определяется в основном плохо обусловленной целевой функцией, сходимость линейная и очень медленная, что соответствует замечанию 5 о методе первого порядка.

Влияние параметра δ. В таблице 2 приведены число итераций и среднее число неравенств во вспомогательной задаче для примеров 4 и 5 при различных δ (при $\delta = \infty$ учитываются все ограничения). Для примера 5 с девятью неравенствами уменьшение δ заметно сокращает размер вспомогательной задачи, но увеличивает число итераций, что согласуется с замечанием 4. В примере 4 единственное ограничение почти всегда δ-активно, и

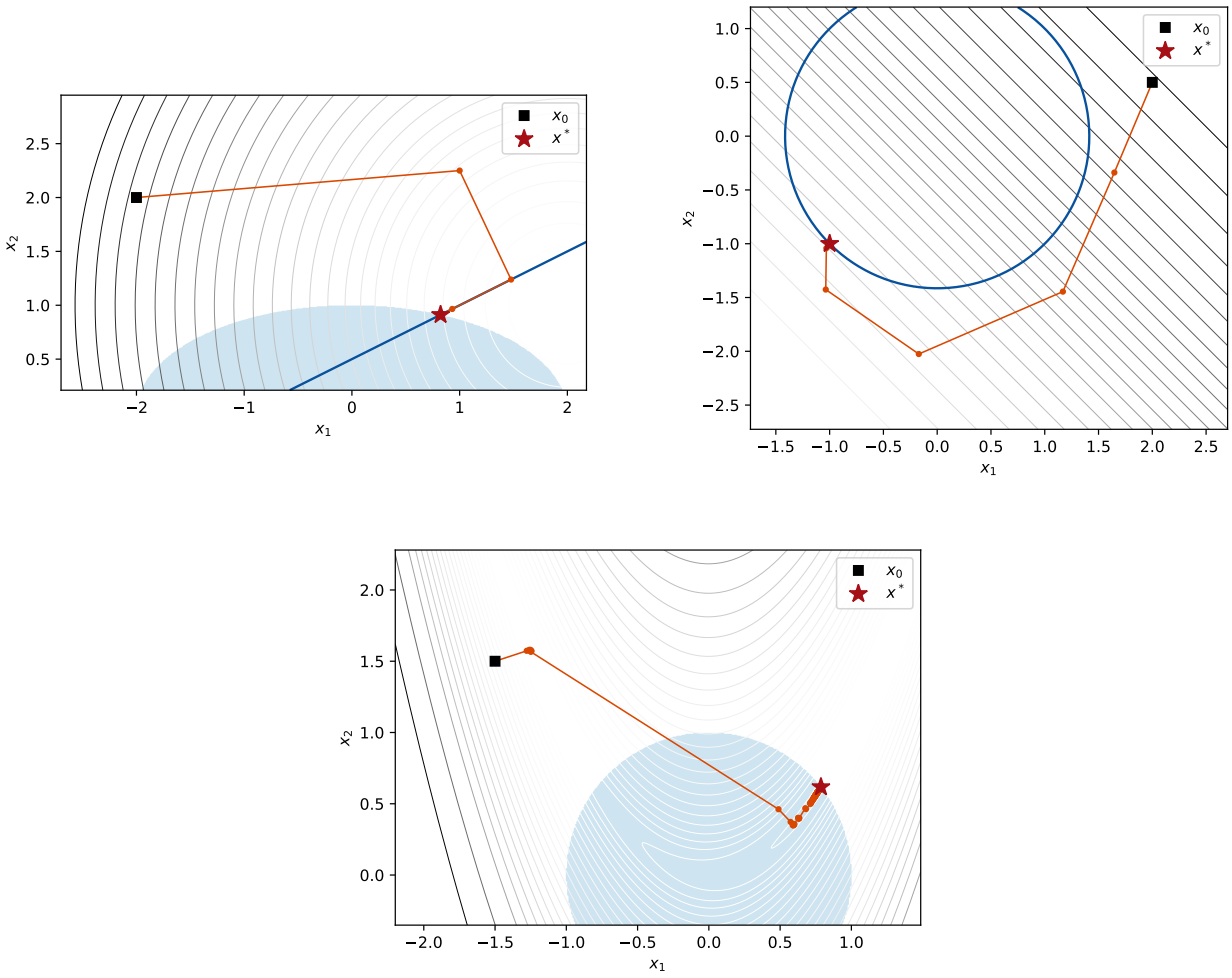


Рис. 1: Траектории метода: пример 2 (слева вверху), пример 3 (справа вверху), пример 4 (внизу)

параметр практически не влияет на работу метода.

Таблица 2: Влияние параметра δ: число итераций / среднее $|I_\delta(x_k)|$

Пример	$\delta = 0{,}1$	$\delta = 1$	$\delta = 10$	$\delta = \infty$
4	2650 / 0,98	2659 / 1,00	2659 / 1,00	2659 / 1,00
5	25 / 1,69	19 / 2,95	10 / 7,45	9 / 9,00

Влияние параметра N. В таблице 3 сравниваются запуски с адаптивным правилом выбора N и с фиксированным N. При фиксированном значении N, меньшем требуемого ($\Lambda_k > N$), направление p_k перестаёт быть направлением убывания Φ_N (лемма 3 неприменима): шаг дробится до $\alpha_{\min}$, и метод останавливается в недопустимой точке, не выполнив критерий остановки за $k_{\max} = 2000$ итераций. Например, в примере 2 при $N = 0{,}1$ метод

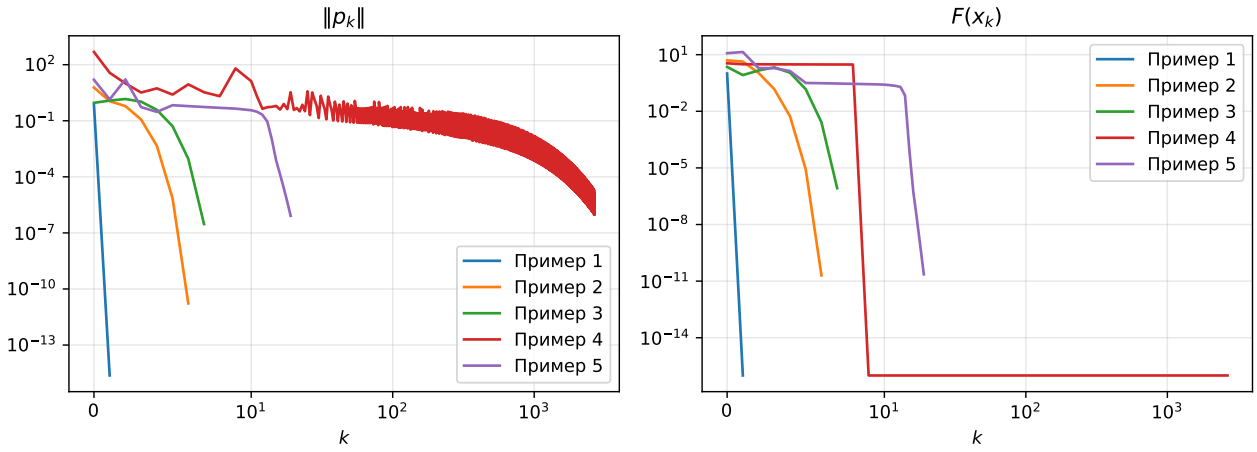


Рис. 2: Величины $\|p_k\|$ и $F(x_k)$ в зависимости от номера итерации

застревает в точке $(1{,}477;\ 1{,}239)$ с $F = 1{,}08$. Адаптивное правило устраняет эту проблему: при малом N_0 итоговое значение параметра оказывается близко к Λ_K, а число итераций почти не зависит от N_0. Завышенное значение N_0 не нарушает сходимости, но может увеличить число итераций (пример 3): при большом N функция Φ_N сильнее штрафует невязку, и шаги, уменьшающие f ценой роста F, чаще отвергаются.

Таблица 3: Число итераций при различных N_0 («—» — критерий остановки не выполнен за 2000 итераций)

	адаптивное N			фиксированное N		
Пример	$N_0 = 0{,}1$	$N_0 = 1$	$N_0 = 10$	$N = 0{,}1$	$N = 0{,}5$	$N = 1$
2	6	6	6	—	—	—
3	7	7	12	—	7	7
5	21	19	13	—	—	—

Заключение

В работе изучен метод линеаризации Б. Н. Пшеничного для задачи нелинейного программирования с ограничениями-неравенствами и равенствами. Приведено обоснование метода: показано, что решение вспомогательной задачи квадратичного программирования задаёт направление убывания функции точного штрафа, равенство нулю этого направления характеризует стационарные точки, а предельные точки последовательности,

построенной методом, при естественных предположениях являются стационарными точками задачи.

Метод реализован на языке Python; вспомогательная задача сводится к задаче о наименьшем расстоянии и решается конечным алгоритмом Лоусона — Хенсона для задачи NNLS, что одновременно даёт направление и оценки множителей Лагранжа. Эксперименты на пяти тестовых задачах, в том числе с недопустимой начальной точкой, невыпуклым допустимым множеством и задачей HS71, подтвердили сходимость метода к стационарным точкам; результаты совпали с результатами библиотечной функции `scipy.optimize.minimize`. Эксперименты также показали, что адаптивный выбор параметра штрафа N существенен для работы метода, параметр δ позволяет уменьшать размер вспомогательных задач ценой увеличения числа итераций, а для плохо обусловленных задач метод, как метод первого порядка, сходится медленно.

A Приложение

A.1 Модуль `linearization.py`

```python
# Метод линеаризации для задачи
#   f(x) -> min,  g_i(x) <= 0 (i < m),  g_i(x) = 0 (i >= m).
from dataclasses import dataclass, field

import numpy as np
from scipy.optimize import nnls


@dataclass
class Problem:
    f: callable       # целевая функция
    grad_f: callable  # её градиент
    g: callable       # вектор ограничений (g_1, ..., g_s)
    jac_g: callable   # матрица Якоби ограничений, размер s x n
    m: int            # первые m ограничений -- неравенства, остальные -- равенства
    name: str = ""


@dataclass
class Result:
    x: np.ndarray
    lam: np.ndarray   # множители при неравенствах
    mu: np.ndarray    # множители при равенствах
    N: float
    iterations: int
    converged: bool
    history: list = field(default_factory=list)


def violation(gx, m):
    # F(x): насколько сильно нарушены ограничения (0, если x допустима)
    return max(0.0, np.max(gx[:m], initial=0.0), np.max(np.abs(gx[m:]), initial=0.0))


def penalty(problem, x, N):
    # функция штрафа Phi_N(x) = f(x) + N F(x)
    return problem.f(x) + N * violation(problem.g(x), problem.m)


def solve_ldp(G, h):
    # Ближайшая к нулю точка множества {z : G z >= h}.
    # Сводится к задаче min |E u - e|, u >= 0 (NNLS, Лоусон -- Хенсон).
    n = G.shape[1]
    if G.shape[0] == 0:
        return np.zeros(n), np.zeros(0)
    E = np.vstack([G.T, h])
    e = np.zeros(n + 1)
    e[-1] = 1.0
    u, _ = nnls(E, e, maxiter=50 * E.shape[1])
    r = E @ u - e
    if np.linalg.norm(r) < 1e-12:
        raise ValueError("линеаризованные ограничения несовместны")
    w = u / (-r[-1])        # множители Лагранжа
    return G.T @ w, w       # z = G^T w
```

```
55
56
57  def direction(c, A_in, b_in, A_eq, b_eq):
58      # Вспомогательная задача:
59      #   <c, p> + |p|^2 / 2 -> min,  b_in + A_in p <= 0,  b_eq + A_eq p = 0.
60      # После замены z = p + c это поиск ближайшей к нулю точки,
61      # равенство записываем как два неравенства.
62      k, q = A_in.shape[0], A_eq.shape[0]
63      G = np.vstack([-A_in, -A_eq, A_eq])
64      h = np.concatenate([b_in - A_in @ c, b_eq - A_eq @ c, A_eq @ c - b_eq])
65      z, w = solve_ldp(G, h)
66      p = z - c
67      lam = w[:k]
68      mu = w[k:k + q] - w[k + q:]
69      return p, lam, mu
70
71
72  def linearization(problem, x0, N=1.0, delta=1.0, eps=0.5, tau=1e-6,
73                    max_iter=10000, min_step=1e-12, adapt_N=True):
74      x = np.asarray(x0, dtype=float).copy()
75      m = problem.m
76      history = []
77
78      for k in range(max_iter):
79          fx, gx = problem.f(x), problem.g(x)
80          df, J = problem.grad_f(x), problem.jac_g(x)
81          F = violation(gx, m)
82
83          # берём только неравенства, близкие к самому нарушенному (delta-активные)
84          active = np.flatnonzero(gx[:m] >= F - delta)
85
86          # направление p и множители из вспомогательной задачи
87          p, lam_act, mu = direction(df, J[active], gx[active], J[m:], gx[m:])
88          lam = np.zeros(m)
89          lam[active] = lam_act
90          Lam = lam.sum() + np.abs(mu).sum()
91          p_norm = np.linalg.norm(p)
92
93          rec = dict(k=k, x=x.copy(), f=fx, F=F, p_norm=p_norm,
94                     active=len(active), Lam=Lam, N=N, alpha=None)
95          history.append(rec)
96
97          # остановка: p почти нулевое и точка почти допустима
98          if p_norm <= tau and F <= tau:
99              return Result(x, lam, mu, N, k, True, history)
100
101         # если N мало, p может не уменьшать штраф -- увеличиваем N
102         if adapt_N and Lam > N:
103             N = max(2.0 * N, Lam)
104             rec["N"] = N
105
106         # дробим шаг, пока штраф не уменьшится достаточно
107         phi = fx + N * F
108         alpha = 1.0
109         while penalty(problem, x + alpha * p, N) > phi - eps * alpha * p_norm ** 2:
110             alpha /= 2.0
111             if alpha < min_step:
112                 break
113         rec["alpha"] = alpha
```

```
114
115         x = x + alpha * p
116
117     return Result(x, lam, mu, N, max_iter, False, history)
```

A.2 Модуль problems.py

```
1  # Тестовые задачи. Ограничения: сначала m неравенств g_i <= 0, затем равенства.
2  import numpy as np
3
4  from linearization import Problem
5
6
7  def example_quadratic():
8      # x1^2 + x2^2 -> min,  1 - x1 - x2 <= 0
9      return Problem(
10         f=lambda x: x[0] ** 2 + x[1] ** 2,
11         grad_f=lambda x: 2 * x,
12         g=lambda x: np.array([1 - x[0] - x[1]]),
13         jac_g=lambda x: np.array([[-1.0, -1.0]]),
14         m=1, name="квадратичная, линейное неравенство")
15
16
17 def example_ellipse():
18     # (x1 - 2)^2 + (x2 - 1)^2 -> min,  x1^2/4 + x2^2 - 1 <= 0,  x1 - 2 x2 + 1 = 0
19     return Problem(
20         f=lambda x: (x[0] - 2) ** 2 + (x[1] - 1) ** 2,
21         grad_f=lambda x: np.array([2 * (x[0] - 2), 2 * (x[1] - 1)]),
22         g=lambda x: np.array([x[0] ** 2 / 4 + x[1] ** 2 - 1, x[0] - 2 * x[1] + 1]),
23         jac_g=lambda x: np.array([[x[0] / 2, 2 * x[1]], [1.0, -2.0]]),
24         m=1, name="эллипс и прямая")
25
26
27 def example_circle():
28     # x1 + x2 -> min,  x1^2 + x2^2 = 2
29     return Problem(
30         f=lambda x: x[0] + x[1],
31         grad_f=lambda x: np.array([1.0, 1.0]),
32         g=lambda x: np.array([x[0] ** 2 + x[1] ** 2 - 2]),
33         jac_g=lambda x: np.array([[2 * x[0], 2 * x[1]]]),
34         m=0, name="окружность")
35
36
37 def example_rosenbrock():
38     # функция Розенброка в круге x1^2 + x2^2 <= 1
39     return Problem(
40         f=lambda x: 100 * (x[1] - x[0] ** 2) ** 2 + (1 - x[0]) ** 2,
41         grad_f=lambda x: np.array([-400 * x[0] * (x[1] - x[0] ** 2) - 2 * (1 - x[0]),
42                                    200 * (x[1] - x[0] ** 2)]),
43         g=lambda x: np.array([x[0] ** 2 + x[1] ** 2 - 1]),
44         jac_g=lambda x: np.array([[2 * x[0], 2 * x[1]]]),
45         m=1, name="Розенброк в круге")
46
47
48 def example_hs71():
49     # задача 71 Хока -- Шиттковского:
50     # x1 x4 (x1 + x2 + x3) + x3 -> min,  x1 x2 x3 x4 >= 25,  1 <= xj <= 5,
```

```
    # x1^2 + x2^2 + x3^2 + x4^2 = 40
    def g(x):
        return np.concatenate([[25 - np.prod(x)], 1 - x, x - 5,
                               [np.sum(x ** 2) - 40]])

    def jac_g(x):
        prod_grad = np.array([np.prod(np.delete(x, j)) for j in range(4)])
        return np.vstack([-prod_grad, -np.eye(4), np.eye(4), 2 * x])

    return Problem(
        f=lambda x: x[0] * x[3] * (x[0] + x[1] + x[2]) + x[2],
        grad_f=lambda x: np.array([x[3] * (2 * x[0] + x[1] + x[2]),
                                   x[0] * x[3],
                                   x[0] * x[3] + 1,
                                   x[0] * (x[0] + x[1] + x[2])]),
        g=g, jac_g=jac_g, m=9, name="HS71")
```

A.3 Модуль experiments.py

```
# Эксперименты: решаем тестовые задачи методом линеаризации и проверяем
# ответ библиотечной функцией scipy.optimize.minimize (метод SLSQP).
from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
from scipy.optimize import minimize

import problems
from linearization import linearization

FIG_DIR = Path(__file__).resolve().parent / "figures"

# задача, начальная точка, начальное N
EXPERIMENTS = [
    (problems.example_quadratic, [0.0, 0.0], 2.0),
    (problems.example_ellipse, [-2.0, 2.0], 1.0),
    (problems.example_circle, [2.0, 0.5], 1.0),
    (problems.example_rosenbrock, [-1.5, 1.5], 1.0),
    (problems.example_hs71, [1.0, 5.0, 5.0, 1.0], 1.0),
]


def solve_scipy(problem, x0):
    # SciPy ожидает неравенства в виде c(x) >= 0, поэтому берём -g
    m = problem.m
    cons = [{"type": "ineq", "fun": lambda x: -problem.g(x)[:m],
             "jac": lambda x: -problem.jac_g(x)[:m]}]
    if problem.g(np.asarray(x0, dtype=float)).size > m:
        cons.append({"type": "eq", "fun": lambda x: problem.g(x)[m:],
                     "jac": lambda x: problem.jac_g(x)[m:]})
    res = minimize(problem.f, x0, jac=problem.grad_f, constraints=cons,
                   method="SLSQP", options={"ftol": 1e-12, "maxiter": 1000})
    return res.x


```

```python
def compare_with_scipy():
    print("Сравнение с scipy.optimize.minimize (SLSQP)")
    results = []
    for idx, (make, x0, N0) in enumerate(EXPERIMENTS, start=1):
        problem = make()
        res = linearization(problem, x0, N=N0)
        x_ref = solve_scipy(problem, x0)
        dx = np.linalg.norm(res.x - x_ref)
        df = abs(problem.f(res.x) - problem.f(x_ref))
        ok = res.converged and dx < 1e-5
        print(f"\nПример {idx} ({problem.name}), итераций: {res.iterations}")
        print(f"  наш метод: x = {res.x}, f = {problem.f(res.x):.8f}")
        print(f"  SLSQP:     x = {x_ref}, f = {problem.f(x_ref):.8f}")
        print(f"  |x - x_SLSQP| = {dx:.1e}, |f - f_SLSQP| = {df:.1e}  ->",
              "совпадает" if ok else "НЕ СОВПАДАЕТ")
        print(f"  F(x) = {res.history[-1]['F']:.1e}, N = {res.N:.3g}, "
              f"lambda = {res.lam}, mu = {res.mu}")
        results.append((problem, res))
    return results


def study_delta():
    print("\nВлияние delta: итераций / среднее число неравенств в подзадаче")
    for idx in (4, 5):
        make, x0, N0 = EXPERIMENTS[idx - 1]
        row = []
        for delta in (0.1, 1.0, 10.0, np.inf):
            res = linearization(make(), x0, N=N0, delta=delta)
            row.append(f"{res.iterations} / {np.mean([h['active'] for h in
    res.history]):.2f}")
        print(f"  Пример {idx}:", ",  ".join(row))


def study_N():
    print("\nВлияние N: число итераций ('-' -- не сошёлся за 2000 итераций)")
    for idx in (2, 3, 5):
        make, x0, _ = EXPERIMENTS[idx - 1]
        row = []
        for adapt, values in ((True, (0.1, 1.0, 10.0)), (False, (0.1, 0.5, 1.0))):
            for N0 in values:
                res = linearization(make(), x0, N=N0, adapt_N=adapt, max_iter=2000)
                row.append(str(res.iterations) if res.converged else "-")
        print(f"  Пример {idx}: адаптивное N0 = 0.1, 1, 10: {row[:3]};"
              f"  фиксированное N = 0.1, 0.5, 1: {row[3:]}")


def plot_trajectory(problem, res, idx):
    # линии уровня f, допустимое множество и точки x_k
    xs = np.array([h["x"] for h in res.history])
    lo = np.minimum(xs.min(axis=0), res.x) - 0.7
    hi = np.maximum(xs.max(axis=0), res.x) + 0.7
    X1, X2 = np.meshgrid(np.linspace(lo[0], hi[0], 400), np.linspace(lo[1], hi[1], 400))
    pts = np.stack([X1.ravel(), X2.ravel()], axis=1)
    Z = np.array([problem.f(p) for p in pts]).reshape(X1.shape)
    G = np.array([problem.g(p) for p in pts]).reshape(*X1.shape, -1)
    m = problem.m

    fig, ax = plt.subplots(figsize=(5.5, 4.5))
    ax.contour(X1, X2, Z, levels=np.unique(np.quantile(Z, np.linspace(0.01, 0.95, 25))),
```

```
                cmap="Greys", linewidths=0.6)
    if m > 0:
        feasible = np.all(G[..., :m] <= 0, axis=-1)
        ax.contourf(X1, X2, feasible, levels=[0.5, 1.5], colors=["#9ecae1"], alpha=0.5)
    for i in range(m, G.shape[-1]):
        ax.contour(X1, X2, G[..., i], levels=[0], colors="#08519c", linewidths=1.5)
    ax.plot(xs[:, 0], xs[:, 1], "o-", color="#d94801", ms=3, lw=1)
    ax.plot(*xs[0], "s", color="black", ms=6, label="$x_0$")
    ax.plot(*res.x, "*", color="#a50f15", ms=12, label="$x^*$")
    ax.set_xlabel("$x_1$")
    ax.set_ylabel("$x_2$")
    ax.set_aspect("equal")
    ax.legend(loc="best")
    fig.tight_layout()
    fig.savefig(FIG_DIR / f"traj{idx}.pdf")
    fig.savefig(FIG_DIR / f"traj{idx}.png", dpi=120)
    plt.close(fig)


def plot_convergence(results):
    # |p_k| и F(x_k) по итерациям; нули рисуем на уровне 1e-16
    fig, axes = plt.subplots(1, 2, figsize=(10, 3.8))
    for idx, (_, res) in enumerate(results, start=1):
        k = [h["k"] for h in res.history]
        axes[0].semilogy(k, [max(h["p_norm"], 1e-16) for h in res.history],
    label=f"Пример {idx}")
        axes[1].semilogy(k, [max(h["F"], 1e-16) for h in res.history], label=f"Пример
    {idx}")
    axes[0].set_title(r"$\|p_k\|$")
    axes[1].set_title(r"$F(x_k)$")
    for ax in axes:
        ax.set_xlabel("$k$")
        ax.set_xscale("symlog", linthresh=10)
        ax.grid(alpha=0.3)
        ax.legend()
    fig.tight_layout()
    fig.savefig(FIG_DIR / "convergence.pdf")
    fig.savefig(FIG_DIR / "convergence.png", dpi=120)
    plt.close(fig)


def main():
    np.set_printoptions(precision=6, suppress=True)
    FIG_DIR.mkdir(exist_ok=True)
    results = compare_with_scipy()
    study_delta()
    study_N()
    for idx, (problem, res) in enumerate(results, start=1):
        if len(res.x) == 2:
            plot_trajectory(problem, res, idx)
    plot_convergence(results)


if __name__ == "__main__":
    main()
```

Список литературы

1. *Hock W.*, *Schittkowski K.* Test Examples for Nonlinear Programming Codes. — Berlin : Springer, 1981. — (Lecture Notes in Economics and Mathematical Systems ; 187).
2. *Базара М.*, *Шетти К.* Нелинейное программирование. Теория и алгоритмы. — М. : Мир, 1982.
3. *Васильев Ф. П.* Численные методы решения экстремальных задач. — 2-е изд. — М. : Наука, 1988.
4. *Измаилов А. Ф.*, *Солодов М. В.* Численные методы оптимизации. — 2-е изд., перераб. и доп. — М. : Физматлит, 2008.
5. *Иоффе А. Д.*, *Тихомиров В. М.* Теория экстремальных задач. — М. : Наука, 1974.
6. *Карманов В. Г.* Математическое программирование. — М. : Физматлит, 2000.
7. *Лоусон Ч.*, *Хенсон Р.* Численное решение задач метода наименьших квадратов. — М. : Наука, 1986.